Thursday Workbook: Clean Up a Playlist, Test First

Week 11 · Pair Reasoning · work in pairs · all data is made up

On Tuesday you watched one process from start to finish. Today **you and a partner run the same process yourselves**, on a different problem:

> A playlist has the same song in it more than once. Some listings are spelled differently ("Slow Orbit", "Slow Orbit (Remastered)", "slow orbit"). Write a small program that finds the repeats and keeps one copy of each song.

You will not write all the code by hand. You may use Claude. **But you write the predictions, the counterexample, and the summaries yourselves,** and you run every test yourselves. That is where the learning is.

Words in **bold** are explained the first time they appear and again in the Glossary page. If a word is not explained, that is our mistake: circle it and tell us.

Part A. Before you touch anything

A1. Why write the test first? (two minutes)

You taste a soup *before* you serve it. You press the smoke-alarm button *before* there is a fire. In both cases you check that the thing does its job while it is cheap to find out that it does not.

A **test** is a tiny program that checks one thing about your other program and answers **pass** or **fail**. A **unit test** checks one small piece (usually one function). Writing the test *first* means you decide what "correct" looks like before you are tempted to bend the definition to fit what you built.

A2. Red and green

We run the tests with a tool called **pytest**. It prints one of two things:

- **Red**: at least one test failed. *This is normal and good.* Red means the test can actually notice a problem. A test that has never been red might not be able to fail at all.
- **Green**: every test passed.

Our rhythm for almost every step: **make it red, then make it green.** If a step says "expect red", a red result means you did it right.

A3. The same three beats at every step

Beat	What you do	Why
PREDICT	Write down what you think will happen, *before* running.	Guessing first is how you notice what you did not understand.
RUN	Run `python3 -m pytest -q`. Look at red or green.	The computer is the referee, not you and not Claude.
EXPLAIN	In your own words, say why it came out that way.	If you cannot explain it, you do not own it yet.

Being wrong in PREDICT is fine and useful. Cross out your guess, do not erase it.

A4. Set up (about 5 minutes; do it together)

You need Python 3.9 or newer and `pytest`.

1. Copy the `starter` folder to a new folder, for example `playlist-lab`. Open a terminal in it.
2. Install pytest if you need to: `python3 -m pip install pytest`
3. Check it works: `python3 -m pytest -q` You should see `no tests ran` (or "collected 0 items"). That is correct: there are no tests yet.

What is in the folder: `playlist/track.py` (what a **Track** is: an id, a title, an artist, a length in seconds), `tests/fixtures.py` (made-up sample tracks; called **fixtures** because they are fixed sample data the tests reuse), and `tests/relations.py` (a ready-made helper that hunts for counterexamples; you will meet it in Step 3).

Do not edit `tests/relations.py` or `playlist/track.py`. Everything else you may change.

Paper path. No laptop, or Python will not install? Do every PREDICT and EXPLAIN box on paper, and do the "by hand" check at each step (a small table you fill in). You will still reach the main idea. The facilitator can give you a printed pytest screen to read.

A5. Start your Claude chat with these rules

Open Claude in the `playlist-lab` folder (or paste code back and forth). **Paste this first, once, before Step 1:**

```
You are my coach for a small Python + pytest project in this folder (Python 3.9+).
Rules for the whole session:
1. We work in tiny steps. I will say which step. Do ONLY that step.
2. Test first. When I ask for a test, write the test (and a stub if needed), then STOP.
   Do not make a failing test pass until I ask you to.
3. Never edit tests/relations.py or playlist/track.py. Never change a test just to make
it pass.
4. Keep code short and plain. After the code, explain each new idea in one sentence of
everyday English.
5. All data is made up. Never use real people or real songs.
6. Tell me the exact command to run (python3 -m pytest -q) and what red or green I
should expect.
```

The prompts in the steps below are what you paste *after* that. Fill in the ____ blanks with **your** answers first.

A6. Keep an honest log

When Claude gives you something, you will mark it in the log at the end: **AI proposed** it, and then **we checked it** by running it or working it out on paper. Both columns matter.

Part B. The steps

Step numbers match Tuesday's steps one-for-one (same numbers, same jobs, different problem). Odd/even pairs are red, then green.

Core (aim to finish together): Steps 1 to 7. Steps 8 to 19 are the same moves again; do them if time allows or at home.

BEAT 1. Cut things into bins (Steps 1–3)

Everyday picture. A post office charges by weight bracket: under 1 lb, 1 to under 5 lb, and so on. The only hard question is the *edge*: a parcel of exactly 1 lb, which bracket? Bugs live at the edges.

A **bin** (also called a **partition** cell) is a group of inputs the function treats the same way. The **boundary** is the pair of values on either side of the edge between two bins. Test one value from the middle of each bin, and test both sides of every boundary.

Step 1 (RED). Write the test for `length_label`

Goal. A function `length_label(seconds)` labels a track by length:

Label	Seconds
`"short"`	0 up to but not including 180
`"normal"`	180 up to but not including 300
`"long"`	300 up to but not including 600
`"epic"`	600 up to and including 3600
(error)	below 0 or above 3600: raise `ValueError`

PREDICT (before anything else). Fill in the test values you will use. For each bin, one value from the middle and the two values right at each edge.

> short: middle **_** ***edges*** *and* **_** ***normal: middle*** edges **_** ***and*** *long: middle* **_** ***edges*** and **_** ***epic: middle*** *edges* **_** ***and*** errors: **_** ***and*** *Which value do you think a lazy programmer would get wrong, and why?* **___**

ASK CLAUDE. (Use your numbers.)

```
STEP 1 (red). Create playlist/lengths.py with a stub length_label(seconds) that just
returns None.
Create tests/test_lengths.py with plain assert tests, one test function per label, using
exactly these
lengths: <paste your table of values>. Add one test that -1 and 3601 raise ValueError.
Do NOT write the real function. Stop after the files exist.
```

RUN. `python3 -m pytest -q`

You should see: red. Several failures such as `assert None == 'short'`. That is good: the stub returns nothing, so every test notices.

EXPLAIN. Why is it good that these tests fail right now? **____**

Check: [ ] test file exists [ ] red result [ ] I can point to a test for each edge value **Stuck?** Red output looks like a wall of text? Read only the last line (`N failed`) and the line starting with `E` (it names the surprise). Test passes at Step 1? Then the stub is not a stub; ask Claude to make it return `None` only.

Step 2 (GREEN). Make `length_label` pass

PREDICT. If the real function used `>` where it should use `>=` on the edge at 180, which of your test values would catch it? **__**

ASK CLAUDE.

```
STEP 2 (green). Now write the real length_label in playlist/lengths.py so all tests in
tests/test_lengths.py pass.
Smallest code that works. Do not touch the tests.
```

RUN. `python3 -m pytest -q` **You should see:** green (all passed).

EXPLAIN. In one sentence, what is each bin, in your own words? ____

Check: [ ] green [ ] I did not change any test **Stuck?** Still red? Look at the *first* failing test. Which of your edge values failed? That value tells you which `>` or `>=` is wrong. Ask Claude *why that one value* failed, not for new code.

Step 3 (GREEN from the start). "Same label" cuts the lengths into groups

A **relation** is a yes/no question about two things. Here: "do these two lengths get the same label?" A relation that is **reflexive** (each thing is related to itself), **symmetric** (if a relates to b then b relates to a) and **transitive** (a to b and b to c means a to c) is an **equivalence relation**: it sorts everything into groups with no overlap. "Same label" should be one. We will check, not hope.

This test is **green from the start** (it records something already true), which is called a **characterization test**.

PREDICT. Will it be green or red? _ *Why?* ____

ASK CLAUDE.

```
STEP 3. Create tests/test_length_relation.py. Define same_label(a, b) = length_label(a)
== length_label(b).
Using reflexive_failure, symmetric_failure and transitive_failure from
tests/relations.py, write three tests
that each assert the failure finder returns None, over the list [0, 90, 179, 180, 240,
299, 300, 450, 599, 600, 1800, 3600].
Add a fourth test that the labels cut that list into exactly four bins.
```

RUN. `python3 -m pytest -q` **You should see:** green. A finder returns `None` when it finds *no counterexample*. `None` is the good answer.

EXPLAIN. Pick one length from each bin. Name a pair that is "the same" and a pair that is not. ____

Check: [ ] green [ ] I know what `None` means from a finder **Stuck?** Import error? Run pytest from the `playlist-lab` folder, not a folder inside it.

BEAT 2. The rule that looks right but is not transitive (Steps 4–7)

Everyday picture. Chairs in a row, 4 feet apart. "Sitting near" means within 5 feet. Chair 1 is near chair 2. Chair 2 is near chair 3. But chair 1 is 8 feet from chair 3, so they are *not* near. "Near" does not chain. A rule that does not chain cannot sort people into tidy groups. This is the whole lesson of today.

Step 4 (RED). A first guess at "looks like the same song"

First attempt: two listings **look alike** if they have the same artist (ignore capital letters) and their lengths are within 5 seconds of each other.

PREDICT. Say whether each pair looks alike, then check with Claude's tests.

> artist Zed 100s vs artist Zed 104s: **_** ***artist Zed 100s vs 105s:*** *artist Zed 100s vs 106s:* **_** ***artist zed 100s vs ZED 101s:*** different artists, both 100s: ____

ASK CLAUDE.

```
STEP 4 (red). Create playlist/songs.py with a stub looks_alike(a, b) that returns False.
Create tests/test_looks_alike.py with tests for: same artist 4 seconds apart; exactly 5
apart;
6 apart; capitals in the artist name differing; different artists with the same length.
Build the Tracks inline with Track(id, title, artist, seconds). Do NOT write the real
rule yet.
```

RUN / expect: red, but only *some* tests fail (the ones that expect `True`). The ones that expect `False` already pass because the stub always says `False`. That is normal.

EXPLAIN. Why do some tests pass even though the function does nothing? ____

Check: [ ] red with some passes [ ] my predictions written before running **Stuck?** Count a pair's gap by hand: `abs(100 - 106)` is 6. Over 5, so not alike.

Step 5 (GREEN). Make `looks_alike` real

ASK CLAUDE.

```
STEP 5 (green). Write the real looks_alike in playlist/songs.py: same artist ignoring
capitals, and lengths
within 5 seconds (5 apart still counts). Make tests/test_looks_alike.py pass. Do not
touch the tests.
```

RUN / expect: green.

EXPLAIN. Which of your PREDICT answers were wrong, and why? (If none, say what you were least sure about.) ____

Check: [ ] green [ ] I can state the rule in one sentence **Stuck?** One test still red? Is it the "exactly 5" test? Then the code needs <=, not <.

Step 6 (RED). Find the counterexample yourself

Now the key move. A **witness** is a concrete example that proves a claim. When the claim is "this rule always chains" and the example breaks it, it is a **counterexample**. Before the computer finds one, **you** find one.

`tests/fixtures.py` has ids 1, 2 and 3 saved for three listings of one song, "Slow Orbit" by "Lantern Hill". They are yours to write. The titles differ on purpose ("Slow Orbit", "Slow Orbit (Remastered)", "slow orbit"), and the third artist is written `LANTERN HILL`.

PREDICT. YOU invent the witness. Choose a length in seconds for each listing so that, under "within 5 seconds":

- listing 1 looks alike listing 2, **and**
- listing 2 looks alike listing 3, **but**
- listing 1 does **not** look alike listing 3.

> Listing 1: ***_ s Listing 2:*** *s Listing 3:* ***_ s Check by hand: gap 1→2 =*** (≤ 5?) gap 2→3 = ***_ (≤ 5?) gap 1→3 =*** _ (> 5?) Draw it: three dots labeled 1, 2, 3. Draw an arrow each way for every pair that looks alike. Which arrow is missing? ____

Do not ask Claude for these numbers. Add your three Tracks to `tests/fixtures.py`, following the pattern of `PAPER_MOONS`, and add them to the front of `TRACKS`:

```
SLOW_ORBIT_1 = Track(1, "Slow Orbit", "Lantern Hill", ____)
SLOW_ORBIT_2 = Track(2, "Slow Orbit (Remastered)", "Lantern Hill", ____)
SLOW_ORBIT_3 = Track(3, "slow orbit", "LANTERN HILL", ____)
TRACKS = [SLOW_ORBIT_1, SLOW_ORBIT_2, SLOW_ORBIT_3, PAPER_MOONS, GLASS_HARBOR,
NIGHT_BUS]
```

ASK CLAUDE.

```
STEP 6 (red). In playlist/songs.py add same_song(a, b) that for now just returns
looks_alike(a, b).
Create tests/test_transitivity.py using reflexive_failure, symmetric_failure and
transitive_failure from tests/relations.py
over TRACKS from tests/fixtures.py: three tests, each asserting the failure is None. For
the transitive test,
make the assert message print the ids and seconds of the three tracks it found.
Do not fix anything. Stop.
```

RUN / expect: one failure, the transitive test. The message names three ids and their seconds. Reflexive and symmetric pass.

EXPLAIN. Did pytest find the same three listings you did? Write the sentence: "Listing __ looks alike listing **, and listing __ looks alike listing** , but **____." Then say in plain words why a single rule that only compares two things at a time cannot be trusted to sort into groups. _____**

Check: [ ] I wrote my numbers *before* running [ ] red on transitivity only [ ] I can read my counterexample aloud **Stuck?** Cannot find numbers? Use the chair picture: 100, 104, 108 under a gap of 5. Now pick your own. All three tests pass? Then your numbers do not break chaining; check gap 1 → 3 is over 5.

Step 7 (GREEN). Make "same song" a real equivalence

The repair: stop asking "are these close?" and instead **normalize** each listing to a **key** (a standard form), and say two listings are the same song exactly when their keys are equal. Here the key is the artist and the title in lowercase, with anything in `( )` and any punctuation removed. Equal keys cannot fail to chain, because "equal" always chains.

Tradeoff, stated plainly: the key is stricter. It will not catch a typo in a title. We gave up some cleverness to get a guarantee.

PREDICT. What key do you expect for `"Slow Orbit (Remastered)"` by `"Lantern Hill"`? _ ***For `"slow orbit"` by `"LANTERN HILL"`?*** _ Equal? ____

ASK CLAUDE.

```
STEP 7 (green). In playlist/songs.py add song_key(track): artist lowercase and stripped,
title lowercase with any
(...) removed, punctuation removed, extra spaces collapsed; returns a tuple. Change
same_song to compare song_keys.
Keep looks_alike as it is. In tests/test_transitivity.py also add (a) a test that
looks_alike is STILL not
transitive over TRACKS (so our counterexample stays on record) and (b) a test that the
three Slow Orbit tracks
have the same song_key and Paper Moons does not.
```

RUN / expect: green.

EXPLAIN. In your own words, why is "equal keys" safe where "close lengths" was not? One sentence: ____

Check: [ ] green [ ] the old counterexample is still a test [ ] I can state the tradeoff **Stuck?** `song_key` gives the wrong key? Ask Claude to print `song_key` for your three tracks, and compare with your PREDICT line.

Core done? If so, jump to Part C and write your summary. Then keep going.

BEAT 3. Group by chains (Steps 8–9)

Everyday picture. "Friend of a friend" is in the group: if Ana knows Ben and Ben knows Cy, all three end up in one friend group even if Ana and Cy never met. Chains of matches build groups.

Step 8 (RED). Cluster tracks by chains of `looks_alike`

PREDICT. Using `TRACKS` and `looks_alike`, how many groups, and who is in each? (Use ids.) ____

ASK CLAUDE.

```
STEP 8 (red). Create playlist/grouping.py with a stub cluster_by_chain(items, rel) that
returns each item in its own list.
Create tests/test_grouping.py over TRACKS with looks_alike: the three Slow Orbit tracks
share one cluster;
Paper Moons, Glass Harbor and Night Bus are each alone; every track is in exactly one
cluster.
Do NOT write the real function.
```

RUN / expect: red (the Slow Orbit cluster test fails).

EXPLAIN. Why must it be exactly one cluster per track, never two? ____

Check: [ ] red [ ] my group prediction written first

Step 9 (GREEN). Make clustering real

ASK CLAUDE.

```
STEP 9 (green). Write the real cluster_by_chain in playlist/grouping.py using union-find
(connected components):
if rel(a, b) is True put them in the same cluster, and chains join clusters. Return a
list of lists.
Make tests/test_grouping.py pass. Explain union-find in two sentences of everyday
English.
```

RUN / expect: green. **EXPLAIN.** This clustering used `looks_alike`, the rule that failed to chain. Why does clustering by chains still give clean groups? ____ **Stuck?** Re-read the friend-of-a-friend picture.

BEAT 4. Is "duplicate" the same both ways? (Steps 10–11)

Everyday picture. "Is the sibling of" works both ways. "Is the boss of" does not. A relation that works both ways is **symmetric**. "Duplicate" should be symmetric: if A is a duplicate of B, B is a duplicate of A. But careless wording ("B is a repeat of A, because B came later") is directional.

Step 10 (RED). `is_duplicate` must be symmetric

PREDICT. If the rule is "same song, and B was added after (or at the same time as) A", is `is_duplicate(1, 2)` true? And `is_duplicate(2, 1)`? _ _ Symmetric?: ____

ASK CLAUDE.

```
STEP 10 (red). Create playlist/duplicates.py with is_duplicate(a, b) = same_song(a, b)
and b.id >= a.id.
Create tests/test_duplicates.py using reflexive_failure and symmetric_failure over
TRACKS, each asserting None;
the symmetric message must print both ids and say which direction is True. Do not fix
the rule.
```

RUN / expect: red on symmetric only; the message shows `is_duplicate(1, 2)` True but `is_duplicate(2, 1)` False.

EXPLAIN. Write the counterexample in one sentence in your own words. ____

Step 11 (GREEN). Make it symmetric

ASK CLAUDE.

```
STEP 11 (green). Fix is_duplicate so swapping a and b can never change the answer. Which
copy to KEEP is a separate
question that we handle later. Make tests/test_duplicates.py pass.
```

RUN / expect: green. **EXPLAIN.** Why is "which one do I keep?" a different question from "are these duplicates?" ____

BEAT 5. Ties and order (Steps 12–13)

Everyday picture. A photo-finish: two runners have exactly the same time. "A finished at or before B" is true *both ways* for two different runners. A proper order, **antisymmetric** (if A is at-or-before B *and* B is at-or-before A, they must be the same thing), needs a tiebreaker such as bib number. An order that is reflexive, antisymmetric and transitive is a **partial order**.

Step 12 (RED). Ties break the order

PREDICT. Which two tracks in `TRACKS` have exactly the same length, so a length-only ordering cannot tell them apart? ids _ ***and*** _

ASK CLAUDE.

```
STEP 12 (red). Create playlist/ordering.py with compare(a, b) returning -1, 0 or 1 by
comparing seconds only.
Create tests/test_ordering.py: define at_or_before(a, b) = compare(a, b) <= 0; test
antisymmetric_failure is None
(message names the two different tracks), transitive_failure is None, and that sorting
every shuffle of TRACKS
(itertools.permutations, functools.cmp_to_key) gives exactly one answer. Do not fix it.
```

RUN / expect: red on antisymmetric and on "one answer". Did the two ids match your prediction?

EXPLAIN. Why does sorting give different results depending on how the list started? ____

Step 13 (GREEN). Add a tiebreaker

ASK CLAUDE.

```
STEP 13 (green). Make compare use a unique sort key: (seconds, artist lowercase, title
lowercase, id).
compare may return 0 only for the very same track. Make tests/test_ordering.py pass.
```

RUN / expect: green. **EXPLAIN.** Why does adding `id` fix it? ____

BEAT 6. Put the pieces together (Steps 14–15)

Everyday picture. After you have tested the screws and the planks, you can build the shelf.

Step 14 (RED). `clean_playlist`

Rule: keep one track per song, the earliest added (smallest id), and return them in id order.

PREDICT. What does `clean_playlist(TRACKS)` return? List the ids. __ And if you run it on its own result? __

ASK CLAUDE.

```
STEP 14 (red). Create playlist/cleaner.py with a stub clean_playlist(tracks) that
returns list(tracks).
Create tests/test_cleaner.py: TRACKS gives the earliest listing of each song in id
order; a playlist with no
duplicates comes back unchanged; an empty playlist stays empty; shuffled input gives the
same result; cleaning
twice gives the same answer as cleaning once. Do NOT write the real function.
```

RUN / expect: red (the duplicate and shuffle tests fail; the "no duplicates", "empty" and "twice" tests already pass with the stub). **EXPLAIN.** Which tests passed with the stub, and what does that tell you about them? ____

Step 15 (GREEN). Make it real

ASK CLAUDE.

```
STEP 15 (green). Write the real clean_playlist using song_key to group the tracks and
keep the earliest id of each group.
Make tests/test_cleaner.py pass.
```

RUN / expect: green. **EXPLAIN.** Which earlier step made this function easy to write? ____

BEAT 7. What depends on what? (Steps 16–19)

Everyday picture. Socks before shoes. Shirt before jacket. Some things must come first; some do not care about each other. And a **cycle** is a trap: "the key is in the room, and I need the key to enter the room." A "needs" order has no cycles. Test the pieces nothing else needs *first*; the idea is "leaves first".

Do this on paper first. Here are our functions and what each one *uses*: `clean_playlist` uses `song_key`; `is_duplicate` uses `same_song`; `same_song` uses `song_key`; `song_key`, `looks_alike`, `cluster_by_chain`, `compare` and `length_label` use no other function of ours. Draw an arrow from each function to what it needs. **Which functions would you test first? __**

Step 16 (RED). Order the tests leaves-first

ASK CLAUDE.

```
STEP 16 (red). Create playlist/depends.py with a CycleError exception and a stub
leaves_first(graph) returning list(graph).
Create tests/test_depends.py with this graph written dependents first:
{"clean_playlist": {"song_key"},
"is_duplicate": {"same_song"}, "same_song": {"song_key"}, "song_key": set(),
"looks_alike": set(),
"cluster_by_chain": set(), "compare": set(), "length_label": set()} where "X": {"Y"}
means "X needs Y".
Tests: every function comes after the ones it needs; every function appears exactly
once. Do not write the real one.
```

RUN / expect: red. **EXPLAIN.** Compare Claude's order with your paper answer. ____

Step 17 (GREEN). Make it real

ASK CLAUDE.

```
STEP 17 (green). Write leaves_first as a depth-first walk: visit what a function needs
first, then add the function.
Mark each function as seen so it is added once. Make tests/test_depends.py pass.
```

RUN / expect: green.

Step 18 (RED). A cycle should be an error

PREDICT. For `{"a": {"b"}, "b": {"a"}}` what should the order be? Is there a right answer? _____

ASK CLAUDE.

```
STEP 18 (red). Create tests/test_cycles.py: leaves_first({"a": {"b"}, "b": {"a"}})
raises CycleError; and for
{"a": {"b"}, "b": {"c"}, "c": {"a"}, "d": set()} the message contains "a -> b -> c ->
a". Do not fix it.
```

RUN / expect: red (no error raised yet).

Step 19 (GREEN). Detect the cycle

ASK CLAUDE.

```
STEP 19 (green). Make leaves_first raise CycleError, naming the loop, when functions
need each other. Track the current path while walking. Make all tests pass.
```

RUN / expect: green, all tests. **EXPLAIN.** What does a cycle in a "needs" order tell you about the design? ____

Part C. Your summary and evidence

C1. Your One-Sentence Summary (write this yourself, no Claude)

Use this pattern, in your own words:

> "I showed that __ **is / is not an equivalence relation, because** _ ***(the counterexample:*** ___), and I fixed it by _____."

C2. The honesty log

Step	What Claude proposed (AI proposed)	How we checked it (ran it / worked by hand)	Did the check agree?

C3. Evidence sheet for your pair (one page, both names)

- Last step we finished: ____ (core is 7)
- Paste or photograph the **red** output at Step 6 and the **green** output at Step 7.
- Our counterexample in words: ____
- One prediction we got wrong, and why: ____
- One thing we would change about our rule next time: ____

C4. Before you leave: the checklist

- [ ] I predicted before every run.

- [ ] I can say what red means and why it can be good.
- [ ] I wrote the counterexample myself.
- [ ] I can say why "close" is not an equivalence and "equal key" is.
- [ ] I marked which things Claude proposed and which we verified.
- [ ] I wrote my own One-Sentence Summary.